

existing optimisation workflow.

A strong educational and research workflow: The framework can draw circuits, inspect operations, simulate ideal or noisy execution and calculate gradients without requiring access to physical hardware. It is therefore useful in classrooms, notebooks, and early-stage research, where understanding the model is more important than immediately deploying it on a quantum processor.

A comparison with other quantum frameworks

No framework is universally best. The useful comparison is not whether one can create a circuit—they all can—but which workflow receives first-class attention (Table 1).

Qiskit is broader than PennyLane and is the natural entry point to IBM's ecosystem. Cirq exposes circuit and device details suited to NISQ

experiments. TensorFlow Quantum builds hybrid models by bringing Cirq circuits into TensorFlow. PennyLane is particularly attractive when the central requirement is end-to-end optimisation of a quantum circuit together with classical ML components.

Important use cases of PennyLane are:

- Quantum neural networks for classification and regression experiments.
- Variational quantum algorithms in which gate parameters are iteratively optimised.
- Quantum kernels and feature maps for exploring alternative representations of data.
- Quantum chemistry, including molecular Hamiltonians and variational energy estimation.
- Combinatorial optimisation prototypes using hybrid objective

functions.

- Generative and probabilistic quantum models.
 - Education, circuit visualisation, gradient demonstrations and backend comparison.
- These are research and experimentation use cases. Present-day QML should not be presented as a guaranteed replacement for classical machine learning. Small demonstrations are valuable for learning the workflow and evaluating hypotheses, but performance, noise, data-loading cost, and fair classical baselines remain essential considerations.

The Iris dataset experiment

The Iris dataset contains 150 samples, four numerical measurements and three flower species. To keep the first experiment easy to inspect, we use 100 samples from two species—Setosa and Versicolor—and two features: petal length and petal width. Seventy-five samples are used for training and 25 for testing. One feature is angle-encoded on each of the two qubits.

The circuit uses feature rotations, two trainable entangling layers and measurements from both qubits. PennyLane converts the circuit into a PyTorch layer. A classical linear layer and sigmoid output then produce the Versicolor probability.

The setup begins by creating and activating a dedicated Python virtual environment, which keeps the experiment and its dependencies isolated from other projects as shown in Figure 2. PennyLane, PyTorch, Scikit-learn and Matplotlib are then installed using pip as shown in Figure 3. These libraries provide the tools required for quantum-circuit construction, model training, dataset preparation, and result visualisation.

Preparing the Iris dataset

The program loads Iris from Scikit-learn, keeps Setosa and Versicolor, selects petal length and petal width, and

```
C:\Users\Admin-SSD>mkdir pennylane-iris
C:\Users\Admin-SSD>cd pennylane-iris
C:\Users\Admin-SSD\pennylane-iris>python -m venv .venv-iris
C:\Users\Admin-SSD\pennylane-iris>dir
Volume in drive C has no label.
Volume Serial Number is 92EE-A284

Directory of C:\Users\Admin-SSD\pennylane-iris

09/02/2026  06:51 PM  <DIR>          .
09/02/2026  06:51 PM  <DIR>          ..
09/02/2026  06:51 PM  <DIR>          .venv-iris
                   0 File(s)              0 bytes
                   3 Dir(s)  9,399,570,432 bytes free

C:\Users\Admin-SSD\pennylane-iris>.venv-iris\Scripts\activate
```

Figure 2: Python virtual environment creation

```
(.venv-iris) C:\Users\Admin-SSD\pennylane-iris>python -m pip install pennylane torch scikit-learn matplotlib
Collecting pennylane
  Downloading pennylane-0.45.1-py3-none-any.whl.metadata (10 kB)
Collecting torch
  Downloading torch-2.13.0-cp312-cp312-win_amd64.whl.metadata (39 kB)
Collecting scikit-learn
  Downloading scikit_learn-1.9.0-cp312-cp312-win_amd64.whl.metadata (11 kB)
Collecting matplotlib
  Downloading matplotlib-3.11.1-cp312-cp312-win_amd64.whl.metadata (80 kB)
Collecting scipy (from pennylane)
  Downloading scipy-1.18.1-cp312-cp312-win_amd64.whl.metadata (61 kB)
Collecting networkx (from pennylane)
  Using cached networkx-3.6.1-py3-none-any.whl.metadata (6.8 kB)
Collecting rustworkx>=0.14.0 (from pennylane)
  Downloading rustworkx-0.18.1-cp310-abt3-win_amd64.whl.metadata (10 kB)
Collecting autograd<1.9 (from pennylane)
  Downloading autograd-1.8.0-py3-none-any.whl.metadata (7.5 kB)
Collecting appdirs (from pennylane)
  Downloading appdirs-1.4.4-py2.py3-none-any.whl.metadata (9.0 kB)
Collecting autoray==0.8.4 (from pennylane)
```

Figure 3: PennyLane installation